NoiseThresholdPrivacyMap<L02I<AbsoluteDistance<UBig», Approximate<ZeroConcentratedDivergence» for ZExpFamily<2>

Michael Shoemate (@Shoeboxam)

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of the implementation of NoiseThresholdPrivacyMap for ZExpFamily<2> in mod.rs at commit f5bb719 (outdated[1]).

1 Hoare Triple

Precondition

Compiler-verified

NoiseThresholdPrivacyMap is parameterized as follows:

- MI, the input metric, is of type L02I<AbsoluteDistance<UBig»
- MO, the output measure, is of type Approximate<ZeroConcentratedDivergence>

Caller-verified

None

Pseudocode

```
# analogous to impl NoiseThresholdPrivacyMap<L02I<AbsoluteDistance<RBig>>, Approximate<
    ZeroConcentratedDivergence>> for ZExpFamily<2> in Rust
class ZExpFamily2:
    def noise_threshold_privacy_map(
        self,
        _input_metric: L02InfDistance[AbsoluteDistance[RBig]],
        output_measure: Approximate[ZeroConcentratedDivergence],
        threshold: UBig,
    ) -> PrivacyMap[
        L02InfDistance[AbsoluteDistance[RBig]], Approximate[ZeroConcentratedDivergence]
    ]:
        #
        noise_privacy_map = self.noise_privacy_map(
            L2Distance.default(), output_measure[0]
        )
        scale = self.scale
```

[1] See new changes with `git diff f5bb719..00154e4 rust/src/measurements/noise_threshold/distribution/gaussian/mod.rs`

```
16
17          def privacy_map(d_in: tuple[u32, RBig, RBig]):
18              l0, l2, li = d_in
19
20              li_sign, li = li.floor().into_parts()  #
21              if li_sign != Sign.Positive:  #
22                  raise f"l-infinity sensitivity ({li}) must be non-negative"
23
24              l2 = l2.min(
25                  li * RBig.try_from(f64.from_(l0).inf_sqrt())
26              )  #
27              li = li.min(l2.floor())  #
28              rho = noise_privacy_map.eval(l2)  #
29
30              if l2.is_zero():  #
31                  return 0.0, 0.0
32
33              if scale.is_zero():  #
34                  return f64.INFINITY, 1.0
35
36              if li > threshold:  #
37                  raise f"threshold must not be smaller than {li}"
38
39              d_instability = threshold - li  #
40
41              delta_single = conservative_discrete_gaussian_tail_to_alpha(
42                  scale,
43                  d_instability,
44              )
45
46              delta_joint: f64 = (1.0).inf_sub(
47                  (1.0).neg_inf_sub(delta_single).neg_inf_powi(IBig.from_(l0)),
48              )
49
50              # delta is only sensibly at most 1
51              return rho, delta_joint.min(1.0)
52
53          return PrivacyMap.new_fallible(privacy_map)
```

Postcondition

Theorem 1.1. Given a distribution `self`, returns `Err(e)` if `self` is not a valid distribution. Otherwise the output is `Ok(privacy_map)` where `privacy_map` observes the following:

Define `function(x)` as a function that omits entries with $v_i = 0$, then updates each remaining pair $(k_i, v_i + Z_i)$, where Z_i are iid samples from `self`, and retains pairs for which $v_i + Z_i > T$ in the positive-threshold direction, or $v_i + Z_i < -T$ in the negative-threshold direction, where $T = |\texttt{threshold}|$. The ordering of returned pairs is independent from the input ordering.

For every pair of elements x, x' in `MapDomain<AtomDomain<TK>, AtomDomain<IBig>>`, and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for `output_measure`, if x, x' are `d_in`-close under `input_metric`, `privacy_map(d_in)` does not raise an exception, and `privacy_map(d_in)` $\leq$ `d_out`, then `function(x)`, `function(x')` are `d_out`-close under `output_measure`.

Proof. Line 11 rejects `self` if `self` does not represent a valid distribution, satisfying the error conditions of the postcondition.

We now construct the privacy map. The `li` sensitivity can be floored, as neighboring integer datasets always differ in whole integer increments. This doesn't affect the check on line 21 to ensure `li` is non-negative.

There are two ways to bound the l2 sensitivity given the three bounds:

1. The `l2` bound directly.

2. Define $x \sim x'$ as $||x - x'||_0 \leq \texttt{l0}$ and $||x - x'||_\infty \leq \texttt{li}$. Then

$$\max_{x \sim x'} ||x - x'||_2 = \max x \sim x' \sqrt{\sum_i^n (x_i - x'_i)^2} \leq \sqrt{\texttt{l0} \cdot \texttt{li}^2} = \sqrt{\texttt{l0}} \cdot \texttt{li} \tag{1}$$

Line 26 updates `l2` to the tighter of these bounds.

`li` similarly has multiple bounds:

1. The `li` bound directly.
2. Define $x \sim x'$ as $||x - x'||_2 \leq \texttt{l2}$. Then

$$\max_{x \sim x'} ||x - x'||_\infty = \max x \sim x' \max_i |x_i - x'_i| \leq \max_i [0, \cdots, \lfloor \texttt{l2} \rfloor, \cdots, 0] = \lfloor \texttt{l2} \rfloor \tag{2}$$

Line 27 updates `li` to the tighter of these bounds.

Now the `l2` sensitivity is zero if any of the three bounds are zero, due to the tightening of the `l2` bound on line 26. The call to `noise_privacy_map` on line 28 rejects negative `l2` sensitivities, including when the scale is zero. Line 30 then checks for zero sensitivity to return a privacy loss of zero rho, zero delta, because all neighboring datasets have identical nonzero-valued pairs after omitting zeros, and the ordering is randomized.

Otherwise sensitivity is nonzero, so if the scale is zero, the privacy loss is unbounded, shown on line 33.

Now that the edge cases are handled, all sensitivities and scales are finite and strictly positive.

The mechanism that adds noise to the values in the hashmap, as described in the conditions of the postcondition of `NoiseThresholdPrivacyMap`, matches the mechanism that adds noise to a vector, as described in the conditions of the postcondition of `NoisePrivacyMap`. Therefore releasing the values incurs the privacy loss computed by `noise_privacy_map` as defined on line 28, The privacy map also ensures that `l2` is non-negative.

We now focus on computing the remaining privacy loss parameter delta. To ensure that the distance to instability remains positive, the threshold must be at least as large as the sensitivity, as checked on line 36.

It suffices to analyze the positive-threshold direction. For a negative threshold $-T$, the release condition $c_j + Z_j < -T$ is equivalent to $-c_j - Z_j > T$. Since the noise distribution is symmetric, $-Z_j$ has the same distribution as Z_j. Negation also preserves all input sensitivity bounds, so the privacy bound is unchanged.

Adapting the proof from [Rog23] (Theorem 7). Let $\text{supp}(x)$ denote the keys whose values are nonzero, treating missing keys as having value zero. Omitting explicit zero-valued entries does not affect any of the input distances. If a key is in $\text{supp}(x) \setminus \text{supp}(x')$, its value differs from zero and is therefore counted by Δ_0; the same holds with x and x' interchanged. Consider $S = \text{supp}(x) \cap \text{supp}(x')$. Define event E to be any potential outcome of the mechanism for which all labels are in S (where only stable partitions are released). We then lower bound the probability of the mechanism returning an event E. In the following, c_j denotes the exact (possibly signed) value for partition j, and Z_j is a random variable distributed according to `self`.

$$\begin{aligned}
\Pr[E] &= \prod_{j \in \text{supp}(x) \setminus \text{supp}(x')} \Pr[c_j + Z_j \leq T] \\
&\geq \prod_{j \in \text{supp}(x) \setminus \text{supp}(x')} \Pr[\Delta_\infty + Z_j \leq T] \\
&\geq \Pr[\Delta_\infty + Z_j \leq T]^{\Delta_0}
\end{aligned}$$

The probability of returning a set of stable partitions ($\Pr[E]$) is the probability of not returning any of the unstable partitions. We now solve for the choice of threshold T such that $\Pr[E] \geq 1 - \delta$.

$$\begin{aligned}\Pr[\Delta_\infty + Z_j \leq T]^{\Delta_0} &= \Pr[Z_j \leq T - \Delta_\infty]^{\Delta_0} \\ &= (1 - \Pr[Z_j > T - \Delta_\infty])^{\Delta_0}\end{aligned}$$

Let `d_instability` denote the distance to instability of $T - \Delta_\infty$. By the postcondition of `conservative_discrete_gaussian_tail_to_alpha`, the probability that a random noise sample exceeds `d_instability` is at most `delta_single`. Therefore $\delta = 1 - (1 - \texttt{delta_single})^{\Delta_0}$.

The privacy map returns `(rho, delta.min(1))`, as δ is bounded by 1. It is shown that `function`(x), `function`(x') are `d_out`-close under `output_measure`. □

References

[Rog23] Ryan Rogers. A unifying privacy analysis framework for unknown domain algorithms in differential privacy, 2023.